

COSC3081/3082/3015 Advanced Programming for Data Science

Assignment 3: NLP Web-based Data Application

Milestone I: Natural Language Processing

Assessment Type	Group assignment. Submit online via Canvas→ Assignments→ Assignment 3→ Milestone I: Natural Language Processing. Marks are awarded for meeting requirements as closely as possible. Clarifications/updates may be made via announcements/relevant discussion forums.
Due Date	End of Week 10, Please check Canvas for the exact due date and time
Marks	20

1. Overview

Today, many online platforms for cosmetics and beauty products enable customers to browse, compare, and purchase items conveniently without visiting physical stores. These platforms generate large volumes of data, including product prices, customer ratings, purchase indicators, and textual reviews. Analyzing such data is essential for understanding customer behavior and improving business decision-making in the beauty industry.

Customer reviews in this domain often provide detailed insights into product quality, effectiveness, and user experience. Combined with structured attributes such as ratings, price, and purchase behavior, this data offers valuable opportunities for data-driven analysis. Techniques from natural language processing can be applied to extract sentiment and key themes from textual reviews, while structured data can support predictive modeling.

By integrating and analyzing both textual and numerical data, it is possible to build models that predict outcomes such as customer ratings or purchasing behavior. These models can help identify patterns in consumer preferences, evaluate product performance, and support more informed decision-making in e-commerce systems.

This assessment consists of two milestones. The first milestone (NLP) focuses on building a text analytics pipeline, from basic preprocessing (parsing, cleaning, wrangling, ...) to text representing and then developing classification models that predict outcomes such as customer ratings or purchasing behavior for cosmetics and beauty products based on review data. The second milestone builds on this work by selecting one or more of the built models and integrating them into a web-based application. In this stage, you will develop a basic online shopping interface that allows users to browse products and view model-driven insights, such as predicted ratings or likelihood of purchase.

Data source: <https://www.kaggle.com/datasets/jithinanievarghese/cosmetics-and-beauty-products-reviews-top-brands>

Note: The dataset has been modified for the course, hence, it is not in its original state.

2. Learning Outcomes

This assessment relates to the following learning outcomes of the course:

- CLO 4: Pre-process natural language text data to generate effective feature representations;
- CLO 5: Document and maintain an editable transcript of the data pre-processing pipeline for professional reporting.

3. Assessment Details

In this milestone, you are required to pre-process a collection of cosmetics and beauty products reviews, build machine learning models for customer ratings or purchasing behavior (i.e., classifying whether the review represents the recommendation of buying a item or not), and perform evaluation and analysis on the built models.

The Data

In this assignment, you are given a collection of cosmetics and beauty products (approx 161200 reviews). The data folder is available to download from Canvas. The three features that you will use in this assignment are: “**review_title**” (title of the review), “**review_text**” (the detailed review of the cosmetics/beauty product), and “**is_a_buyer**” (label represent that the user actually buy the cosmetics/beauty product or not. ‘*True*’ and ‘*False*’ represent “*buy*” and “*not buy*” respectively).

Task 1: Basic Text Pre-processing [4 marks]

In this task, you are required to perform basic text pre-processing on the given dataset, including, but not limited to tokenization, removing most/least frequent words and stop words. In this task, we focus on pre-processing the “**Review Text**” only. You are required to perform the following:

1. Extract information about the review, and then perform the pre-processing steps mentioned below to the extracted reviews
2. Tokenize each cosmetics/beauty review. The word tokenization must use the following regular expression, **r"[a-zA-Z](?:[a-zA-Z])?"**.
3. All the words must be converted into the lower case.
4. Remove words with a length less than 2.
5. Remove stopwords using the provided stop words list (i.e., **stopwords_en.txt**). It is located inside the same downloaded folder.
6. Remove the word that appears only once in the document collection, based on **term** frequency.
7. Remove the top 20 most frequent words based on **document** frequency.
8. Save the processed data as **processed.csv** file.
9. Build a vocabulary of the cleaned/processed reviews, and save it in a txt file (please refer to the Required Output section);

Note:

- For all the words that we removed (including step 4,5,6,7), you will also exclude them in the generated vocabulary.
- The output of this task will be checked against the expected output. You should strictly follow the following format requirement.

Required Output for Task 1:

The output of this task must contain the following files:

- **processed.csv** file
- **vocab.txt** This file contains the **unigram** vocabulary, one each line, in the following format: **word_string:word_integer_index**. Very importantly, **words in the vocabulary must be sorted in alphabetical order, and the index value starts from 0**. This file is the key to interpret the sparse encoding. For instance, in the following example, the word **aaron** is the 20th word (the corresponding integer_index as **19**) in the vocabulary (note that the index values and words in the following image are artificial and used to demonstrate the required format only, it doesn't reflect the values of the actual expected output).

```
aand:12
aantrekkelijk:13
aap:14
aar:15
aarca:16
aardman:17
aareon:18
aaron:19
aaronwallis:20
aarolette:21
aarolettehotel:22
```

Fig.1 example format for vocab.txt

- All Python code related to Task 1 should be written in the jupyter notebook **task1.ipynb**.

Task 2: Generating Feature Representations for Cosmetics/Beauty Reviews [7 marks]

In this task, you are required to generate different types of feature representations for the cosmetics/beauty item reviews. Note that in this task, we will only consider the **description/text** of the review (ignore *Title*). The feature representation that you need to generate includes the following:

Bag-of-words model:

- Generate the **Count** vector representation for each cosmetics/beauty review, and save them into a file (please refer to the required output). Note, the generated Count vector representation must be based on the generated **vocabulary** in Task 1 (as saved in **vocab.txt**).

Models based on word embeddings:

- You are required to generate a feature representation of cosmetics/beauty reviews based on the following language models:
 - Choose 01 embedding language model, e.g., FastText, GoogleNews300, or other Word2Vec pretrained models, or Glove, which will be used to extract the feature vector for each review.
 - You are required to build the weighted (i.e., TF-IDF weighted) and unweighted vector representation for each cosmetics/beauty review using the chosen language model.

To summarise, there are 3 different types of feature representation of documents that you need to build in this task, weighted, unweighted and embedding representation.

Required Output for Task 2:

- **count_vectors.txt** This file stores the sparse count vector representation of cosmetics/beauty reviews in the following format. Each line of this file corresponds to one review. It starts with a '#' key followed by the index of the item review, and a comma ','. The rest of the line is the sparse representation of the corresponding description in the form of **word_integer_index:word_freq** separated by comma. Following is an example of the file format (note that the following image is artificial and used to demonstrate the required format only, it doesn't reflect the values of the actual expected output):

```
#12612628,11:1,35:1,255:4,256:1,411:3,503:1,535:2,615:4,727:2,742:1,752:1,782
:1,833:3,862:1,893:1,941:1,1067:1
#12612830,0:1,64:1,70:1,72:1,90:1,92:1,116:1,218:1,231:1,245:1,270:1,274:1,31
0:1,316:1,317:1,330:2,412:2,433:1,441:1,529:1,532:1,537:1,538:1,545:2,616:2,6
17:1,649:1,680:1,697:1,737:1,742:1,744:1,752:1,766:1,768:1,773:2,774:1,799:1,
845:1,855:1,858:1,864:1,874:1,888:2,902:1,937:1,952:2,962:1,1003:1,1008:1,107
7:1,1082:1,1087:1,1092:1,1093:1
#12612844,6:2,17:1,57:1,59:2,134:2,149:2,169:2,196:1,198:1,199:2,208:3,214:1,
218:1,245:1,252:1,255:6,259:1,265:1,327:2,340:1,352:1,387:2,390:2,432:1,450:1
```

Fig. 4 Example format for count_vectors.txt

Note: **word_freq** here refers to the frequency of the unigram in the corresponding **Review Text** only, excluding the **Review Title**.

- **unweighted_vectors.txt** This file stores the unweight vector representation of cosmetics/beauty reviews in the following format. Each line of this file corresponds to one review. It starts with a '#' key followed by the index of the item review, and a comma ','. The rest of the line is the unweight vector representation of the corresponding description in the form of **sequence of values, separated by comma (",")**. The length of the line depend on the embedding language model that you choose.
- **weighted_vectors.txt** This file stores the weight vector representation of cosmetics/beauty reviews in the following format. Each line of this file corresponds to one review. It starts with

a '#' key followed by the index of the item review, and a comma ','. The rest of the line is the weight vector representation of the corresponding description in the form of *sequence of values, separated by comma (",")*. The length of the line depends on the embedding language model that you choose.

Task 3: Cosmetics/Beauty Products Review Classification [9 marks]

In this task, you are required to build machine learning models for classifying the purchasing behavior for a cosmetics and beauty product based on review data. A simple model that you can consider is the logistic regression model from sklearn as demonstrated in the activities. However, you feel free to select other models (even if it has not been covered in this course). You are required to conduct two sets of experiments on the provided dataset to investigate the following two questions, respectively.

Q1: Language model comparisons [3 marks]

Which language model we built previously (based on product reviews) performs the best with the chosen machine learning model? To answer these questions, you are required to build machine learning models based on the feature representations of the documents you generated in **Task 2**, and to perform evaluation on the various model performance.

Q2: Does more information provide higher accuracy? [6 marks]

In **Task 2**, we have built a number of feature representations of documents based on text review. However, we have not explored other features of an item review, e.g., the title of the review, the price, the average rating... Will adding extra information help to boost up the accuracy of the model? To answer this question, you are required to conduct experiments to build and **compare** the performance of classification models that consider:

- only description/text of the review (which you've already done in Task 3 – Q1)
- (3 marks) description/text and title of the review
- (3 marks) adding extra information of a product such as brand name, product title, average product rating, price... For this task, you have the flexibility to generate the feature representation for the product. You can use only one final feature representation or can use different type of feature representations in the classification models.

Note that: for both questions above,

- Consider demonstrating the comparison using at least three different models: one based on the bag-of-words model, and the other two focusing on weighted and unweighted models.
- When evaluating the performance of the models, you are required to conduct a **5-fold cross validation** to obtain robust comparisons.

All Python code related to Task 2 and 3 should be written in the jupyter notebook task2_3.ipynb.

Marking Guidelines

Marking Criteria

- **Mechanical pass:** Your outputs will be compared against the expected output. Therefore, marking will be based on the similarity between what we expect (as discussed in the instructions) and what we receive from you. It is extremely important to carefully follow the instructions to produce the expected output. Otherwise, you may easily lose many points for simple mistakes (e.g. typos in the format of the files, not loading essential libraries, different file names/path, etc).

- **Expert pass:** Your Jupyter notebook will be checked by an expert to validate the logic and flow, proper use of libraries and functions, and clarity of codes, comments, structure and presentation.
- You need to ensure all the codes and files that are required to run your code are included in the submission. The expert will **NOT** fix your code's problem even if it is a simple typo in a file name or an imported library.

Mark Allocations

- Task 1 Basic Text Pre-processing [4%]
 - Implementation [3%]
 - Notebook presentation [1%], proportional to actual mark obtained in implementation
- Task 2 [7%]
 - Implementation [5%]
 - Notebook presentation [2%], proportional to actual mark obtained in implementation
- Task 3 [9%]
 - Implementation [7%]
 - Notebook presentation [2%], proportional to actual mark obtained in implementation

For Task 1, and Task 2 and 3, you are required to maintain an auditable and editable transcript, and communicate any justification of methods/approach chosen, results, analysis and findings through jupyter notebook. The presentation of the jupyter notebook accounts for certain percentages of the allocated mark for each task, proportional to the actual mark obtained, as per specified above. Students can refer to the activities in modules as examples for the level of details that they should include in their jupyter notebook.

Submission

The final submission of this milestone will consist of:

- The required output from Task 1, including **vocab.txt**
- The required output from Task 2: **count_vectors.txt**, **un_weighted_vectors.txt**, **unweighted_vectors.txt**
- The jupyter notebook of Task 1, and Task 2&3, respectively
- The **.py** format of the jupyter notebook of Task 1, and Task 2&3, respectively. Note that:
 - The content of the .py file must match your jupyter notebook
 - The .py file will be used for plagiarism detections on both comment/description content, as well as the actual code.
 - To help promote academic integrity, please make sure you submit the .py files. Submission without the .py files or unmatched .py files will NOT be marked.
 - Note that the .py files can be easily downloaded from jupyter notebook interface (File - > Download as -> Python (.py))
- Put all the above mentioned files in a folder, named with your student id, zip the folder with the same name (i.e., **s1234567.zip**) and upload for submission

Assessment declaration:

When you submit work electronically, you agree to the assessment declaration:
<https://www.rmit.edu.au/students/student-essentials/assessment-and-exams/assessment/assessment-declaration>

Late Submission Penalty

Late submissions will incur a 10% penalty on the total marks of the corresponding assessment task per day or part of day late. Submissions that are late by 5 days or more are not accepted and will be awarded zero, unless special consideration has been granted. Granted Special Considerations with a new due date set more than 2 weeks after the original due will automatically result in an equivalent assessment in the form of a practical test with interview, assessing the same knowledge and skills of the assignment (location and time to be arranged by the instructor). Please ensure your submission is correct (all files are there, compiles etc), re-submissions after the due date and time will be considered as late submissions.

Academic integrity and plagiarism (standard warning)

Academic integrity is about honest presentation of your academic work. It means acknowledging the work of others while developing your own insights, knowledge and ideas. You should take extreme care that you have:

- acknowledged words, data, diagrams, models, frameworks and/or ideas of others you have quoted (i.e. directly copied), summarised, paraphrased, discussed or mentioned in your assessment through the appropriate referencing methods,
- provided a reference list of the publication details so your reader can locate the source if necessary. This includes material taken from Internet sites.

If you do not acknowledge the sources of your material, you may be accused of plagiarism because you have passed off the work and ideas of another person without appropriate referencing, as if they were your own.

RMIT University treats plagiarism as a very serious offence constituting misconduct. Plagiarism covers a variety of inappropriate behaviours, including:

- Failure to properly document a source
- Copyright material from the internet or databases
- Collusion between students

For further information on our policies and procedures, please refer to <https://www.rmit.edu.au/students/student-essentials/rights-and-responsibilities/academic-integrity>